# Smart Code Tutor

Development of a Multi-Agent System Built on Fine-Tuned CodeT5 for Personalized Code Explanation

CS5202 – Generative AI & Large Language Models

**TEAM – 13**


| **Name** | **ID** |
|---|---|
| Kush Jalan | SE23UCSE244 |
| Vanshika Batra | SE23UMCS062 |


## 1. Introduction

Artificial Intelligence is changing the way people learn programming and understand software systems. One important use of large language models is automatic code explanation means where a model reads source code and explains what it does in simple language.

Models such as CodeT5 have shown strong performance in code understanding and summarization tasks. They can generate accurate explanations but these explanations are often technical and difficult for beginners to understand. Most systems assume prior programming knowledge, which makes learning harder for students and first time programmers.

The Smart Code Tutor project is designed to solve this problem by making code explanations easier to understand. By combining a fine-tuned CodeT5 model with a multi-agent refinement framework, the system aims to generate explanations that are simpler, clearer and better suited to the learner's level of understanding.

The goal of this project is to build a beginner-friendly intelligent programming assistant that supports learning through personalized and structured explanations.

## 2. Domain Research Note

Understanding programming code is one of the biggest challenges faced by beginners when learning software development. Writing code becomes easier with practice but understanding how an existing piece of code works, why a certain logic is used and how different statements connect together is often much more difficult. Many students

struggle not because programming is impossible to learn but because explanations are often presented in a way that assumes prior knowledge.

### 2.1. The Real Problem

The main problem this project focuses on is the gap between technical correctness and learning usefulness. Existing AI systems can generate explanations that are correct but correctness alone is not enough for learning. A beginner needs explanations that are simple, step-by-step and easy to connect with basic programming concepts.

For instance, an advanced programmer may understand a short technical explanation immediately while a beginner may need that same explanation broken down into smaller logical parts. Current models usually do not make that adjustment.

### 2.2. Who This Problem Affects

This issue mainly affects:

- Students learning programming for the first time
- Beginner developers who are building confidence in coding
- Non-technical learners entering software related fields
- Self-learners using online resources without teacher guidance

For these users, technical explanations often create confusion instead of helping understanding. As a result, learning becomes slower and more frustrating.

### 2.3. Why Existing Solutions Fall Short

Modern transformer-based models such as CodeT5 are powerful in code summarization and explanation but they still have several limitations:

- Explanations are often accurate but too technical.
- Most systems follow a one-size-fits-all explanation style.
- They do not adapt explanations for different skill levels.
- Step-by-step reasoning is usually missing.
- Real learning support is limited beyond direct summarization.
- Explanations often lack examples that make understanding easier.

This means current systems work well for developers but not as well for learners.

### 2.4. Why Smart Code Tutor is a Good Approach

Smart Code Tutor aims to solve this by combining two strengths.

First, **fine-tuned CodeT5** will be trained on beginner-friendly explanation pairs so that its output becomes simpler and easier to understand.

Second, a **multi-agent refinement system** will improve explanations further. Different agents can focus on simplifying language, adding examples, improving clarity and refining structure. Instead of giving one direct explanation, the system improves the explanation step by step.

This approach makes explanations more human-like, more educational and better suited for learning.

### 2.5. Expected Impact

If successful, Smart Code Tutor can help beginners understand programming concepts faster, reduce confusion while learning and create a more supportive coding learning experience. It can also act as a foundation for future intelligent tutoring systems in programming education.

## 3. Data Pipeline

A complete data pipeline has been developed to prepare training data for improving explanation quality.

### 3.1. Dataset Creation Approach

The dataset was created using a hybrid strategy:

- Code snippets collected from the CodeSearchNet dataset
- Beginner-friendly explanations generated using Gemini-based assistance
- Manual refinement performed to improve explanation clarity

### 3.2. Data Format

Each dataset sample follows a sequence-to-sequence structure suitable for transformer-based learning models such as CodeT5:

**Input:** Code snippet

**Output:** Beginner-friendly simplified explanation of the code

### 3.3. Processing Steps

The dataset preparation pipeline includes:

- Filtering valid functions from the CodeSearchNet dataset
- Generating simplified explanations automatically
- Cleaning and normalizing explanation text
- Converting samples into structured JSON training format

The processed dataset has been successfully generated and loaded for future fine-tuning.

## 4. Initial Model Running with Preliminary Results

An initial implementation pipeline has been prepared using the Hugging Face Transformers framework for integrating the pretrained CodeT5 model.

At the current stage, the following progress has been achieved:

- CodeSearchNet dataset successfully loaded
- Explanation dataset generated using Gemini-assisted processing
- Structured input-output training pairs created
- Dataset prepared for CodeT5 fine-tuning

### 4.1. Sample Preliminary Results

**Example 1**

**Input Code:**

```
def square(n):
    return n * n
```

**Generated Explanation:**

This function takes a number as input and returns its square by multiplying the number by itself.

**Example 2**

**Input Code:**

```
def add(a, b):
    return a + b
```

**Generated Explanation:**

This function accepts two input values and returns their sum.

**Example 3**

**Input Code:**

```
def is_even(num):
    return num % 2 == 0
```

**Generated Explanation:**

This function checks whether a number is even by verifying if it is divisible by 2.

**Observation**

These early results confirm that the dataset preparation pipeline is successfully generating structured and beginner-friendly explanation pairs.

## 5. Conclusion

The first phase of Smart Code Tutor has helped build a strong foundation for the project. Through domain research we have clearly identified that the biggest gap in current code explanation systems is not correctness but instead making explanations understandable for beginners.

A working data pipeline has been successfully developed and structured beginner-friendly explanation pairs have been generated from source code snippets. The initial results are promising and show that the project is moving in the right direction.

The next phase will focus on fine-tuning CodeT5 and integrating the multi-agent refinement framework so that explanations become clearer, simpler and more personalized for learners.

Eventually, this project can grow beyond a simple explanation system and become a practical intelligent learning assistant that supports students, coding platforms and programming education in a much more meaningful way.